THE OPEN SOURCE AUDIT

A Capstone Project for OSS NGMC Course

Subject: Open Source Software | Units 1–5

Software Chosen: Python

Student Name: Saurav (24BSA10300)

Course: Open Source Software (OSS NGMC)

1. Introduction

Open source is one of those topics I honestly didn't take very seriously before starting this course. I thought it basically just meant free software — like, stuff you didn't have to pay for. Turns out that's a pretty shallow way of looking at it. After going through the units and doing this audit, I'd say open source is more like a whole philosophy about how knowledge gets built and shared. The more I read about it the more I realised how much of what we use everyday is built on open foundations — the servers, the tools, even the operating systems on phones.

I picked Python for this audit because it's the language I use most and I figured it would be easier to understand the technical parts. But also honestly because Python's story is genuinely interesting. It wasn't made by a company — it was made by one guy, Guido van Rossum, who just wanted something cleaner than what was available at the time. The fact that it became one of the most widely used languages in the world is kind of remarkable when you think about it.

This report goes through Python's background, its license, the ethical debates around open source, how Python looks on a Linux system, what its ecosystem looks like, and a comparison with MATLAB as a proprietary alternative. There are five shell scripts included too. I'll be honest — some sections I found more interesting than others. The ethics part especially made me think about things I hadn't really considered before, like who actually pays for the infrastructure everyone relies on.

Part A — Origin and Philosophy

A1. The Problem Python Was Created to Solve

To understand why Python exists, you have to picture what programming was actually like in the late 1980s. This was before the internet was really a thing for regular people, before IDEs were common, before most of the convenience developers take for granted now. If you wanted to write a script to automate something, the main options were C, shell scripting, or languages like Perl and AWK. C required you to manage memory yourself, which is fine if you're building something serious but complete overkill for a quick task. Shell scripting was useful but got really messy once you tried doing anything complex. Perl worked but it looked like someone had typed randomly with their face.

Guido van Rossum was working at CWI in the Netherlands in 1989. He'd been part of the team building a language called ABC, which actually had some decent ideas — clean syntax,

readable code, tried to be friendly to non-experts. The problem was ABC was completely closed off. You couldn't link it to C libraries or extend it in any meaningful way. It did exactly what its designers intended and nothing else. Van Rossum liked the readability of ABC but found the rigidity frustrating.

So over a holiday break in December 1989, he started writing something new. He's said in interviews that he basically needed a project to keep himself busy. The first public version, Python 0.9.0, was posted to a Usenet group (alt.sources) in February 1991. He shared it publicly on purpose — he wanted feedback, he wanted people to use it and break it and tell him what was wrong.

What I find interesting is that Python wasn't created to solve some giant technical problem. It was mostly created because the existing tools were annoying to use. That's a very human reason to build something, and I think it's part of why Python ended up being so accessible — it was designed from the start by someone thinking about the person actually sitting at the keyboard, not just the computer running the code.

The impact ended up going way beyond what van Rossum probably expected. Python became the go-to language for scientists, researchers, data analysts, people who weren't 'real programmers' in the traditional sense. Biologists started using it to process genetic data. Economists used it for modelling. Teachers used it to introduce programming to students. This wouldn't have happened with C.

A2. The License — What It Actually Says

The Python Software Foundation License (PSF License)

Python uses the Python Software Foundation License version 2, usually written as PSF-2.0. You can find the full text at python.org/psf/license and it ships with every Python installation in a file called LICENSE. The OSI (Open Source Initiative) has approved it, and SPDX lists it as PSF-2.0.

The short version: you can use, copy, modify, distribute, and even sell Python or things built on it. The main conditions are just keeping the copyright notice and disclaimer intact. That's basically it. It's a very permissive license, which was a deliberate choice by the PSF.

The Four Freedoms of Free Software

Richard Stallman, who founded the GNU Project, defined what he called the four essential freedoms that determine whether software is genuinely free:

- Freedom 0: Run the program for any purpose.
- Freedom 1: Study how it works and modify it.
- Freedom 2: Redistribute copies.
- Freedom 3: Distribute modified versions.

Python's license covers all four. You can run it commercially, read and modify the CPython source on GitHub, bundle it with products you sell, and fork it into your own version as long as you keep the original copyright notice. There's no copyleft clause requiring you to open-source your own code. No field-of-use restrictions. By Stallman's own definition, Python qualifies as free software.

Can a Company Take Python, Modify It, and Sell It Without Sharing Changes?

Yes, completely. This is the main difference between the PSF license and the GPL. Under PSF, a company can fork CPython, make whatever changes they want, put it inside a proprietary product, and ship it without ever releasing their changes. The only rule is keeping the copyright attribution.

This has actually happened quite a bit. Anaconda distributes a customised Python with proprietary tooling layered on top. IronPython had proprietary components at one point. Various embedded systems companies ship modified Python interpreters and don't release the source. All of it is legal under PSF.

Under the GPL this wouldn't be allowed — you'd have to release your modified source under the same terms. The PSF went with permissiveness intentionally. The thinking was that fewer restrictions = more adoption, and more adoption = a bigger ecosystem. That seems to have worked out given where Python is today.

GPL vs MIT vs PSF — Key Differences

Feature	GPL v2	MIT	PSF (Python)
Can use in commercial software?	Yes	Yes	Yes
Must share source of modifications?	Yes (copyleft)	No	No
Can relicense?	No	Yes	Yes (with conditions)
Patent grant included?	Yes (v3)	No	No
Attribution required?	Yes	Yes	Yes

If I was choosing a license for something I wanted widely adopted with minimum friction, I'd go MIT. It's short, clear, and developers everywhere know what it means. GPL makes sense if you're building infrastructure and genuinely don't want companies taking it proprietary — that copyleft mechanism is actually a strong protection. PSF sits closer to MIT but is specifically tailored for a language runtime rather than a library or application.

Free as in Free Beer vs. Free as in Freedom

Stallman's distinction between these two is one of the most useful framings in this entire field. 'Free as in free beer' means no cost. 'Free as in freedom' means actual rights over the software — to use, study, change, and redistribute it.

Python is both. You can download it from python.org without paying anything, ever. But it's also free as in freedom: the PSF license gives you real control. You're not dependent on a corporation's decisions about where the language should go. If the PSF went in a direction the community hated, they could fork it — PyPy and Jython both exist because that freedom is real, not just theoretical.

The contrast with MySQL is useful here. MySQL is free to download in many cases, but Oracle's ownership created enough uncertainty that the community forked it into MariaDB. Python doesn't have that problem because there's no commercial owner making decisions based on revenue targets.

A3. The Ethics of Open Source

Should All Software Be Open Source?

I went back and forth on this one while writing this section. The idealistic answer is yes — open code means anyone can audit it for security problems, bugs get found faster because more people can look, knowledge isn't locked behind paywalls, and people in countries with less money get access to the same tools as the biggest companies. The public internet runs on open code and that's been genuinely good for the world.

But the practical answer is more complicated. Software costs money to make. Engineers, infrastructure, support, documentation — none of that is free. Medical imaging systems, specialised financial tools, certain industrial software — these need sustained expert investment that volunteers simply won't maintain indefinitely. If you forced everything open you'd reduce the financial incentive to build certain types of niche but important software.

My actual view is that open source makes the most sense for infrastructure — the stuff multiple organisations build on top of. Compilers, databases, operating systems, networking tools. That's where the community model works really well because everyone benefits from improvements. Product software is different — if the software is the product and development cost isn't shared, you need a different model. Both can coexist. I don't think it has to be one or the other.

Is It Ethical for Companies to Build on Free Community Labor?

This is the question I found most uncomfortable to think about. Google, Meta, Amazon, Microsoft — trillion dollar companies, all built on Linux, Python, PostgreSQL, and dozens of other projects that were mostly written by volunteers or underfunded maintainers.

Some companies do give back meaningfully. Red Hat built a real business on Linux and contributed seriously — code, full-time engineers, funding. Google employs Python core developers. These are legitimate examples of the relationship working.

But a lot of companies just… don't. They use open-source dependencies, build products on them, and give nothing back. The Heartbleed bug in 2014 made this really visible — OpenSSL, which was protecting a huge amount of internet traffic and financial transactions, was being maintained by essentially a handful of people with no real resources. That's a failure of the companies that profited from it, not just bad luck.

Using open-source software without contributing back is legal. The license allows it. But I think it reflects pretty poor values. The sustainable way to build on community work is to invest back into that community somehow — code, money, or at minimum hiring people who do contribute. Companies that just extract value without giving anything back are slowly degrading something everyone, including themselves, depends on.

Does Open Source Enable or Undermine Innovation?

It enables it, clearly. The whole point of standing on the shoulders of giants is that you start higher up. When you don't have to rebuild a database, a web server, or a machine learning library from scratch, you can spend your time on the thing that's actually new. NumPy, pandas, scikit-learn, PyTorch — all open, all built on each other, all made it possible for researchers to do actual science instead of numerical computing infrastructure work. The AI developments of the last decade wouldn't have happened at the same speed without open-source foundations.

The 'it undermines innovation' argument only works if you think innovation means never using prior work. That's not how physics works, or architecture, or music. Python itself is built on ABC, C, and Modula-3. Reusing and improving prior work is how knowledge advances in every field.

Part B — Linux Footprint

B1. Installation Method

On most Linux distributions Python comes through the package manager. On Debian/Ubuntu:

```
sudo apt update && sudo apt install python3 python3-pip
```

On Fedora/RHEL/CentOS:

```
sudo dnf install python3
```

You can also compile from source if you need a specific version or build flags that aren't in the package manager. The source is on python.org and GitHub (github.com/python/cpython). You'll need GCC, make, and headers for optional modules like ssl and sqlite. It follows the standard configure/make/make install process, nothing unusual.

B2. Key Directories on a Linux System

Directory / Path	What It's For
/usr/bin/python3	Main interpreter binary — what runs when you type python3
/usr/lib/python3.x/	Standard library (os, sys, json, etc.)
/usr/lib/python3/dist-packages/	System-wide packages installed via apt
/usr/local/lib/python3.x/site-packages/	pip-installed packages (local/root)
~/.local/lib/python3.x/site-packages/	User pip packages (--user flag)
/etc/python3/	System-wide config (varies by distro)
/usr/share/doc/python3/	Documentation and changelog
/var/log/	Where Python apps write logs

The split between /usr/lib (system packages managed by apt/dnf) and /usr/local/lib (pip installs) is important. It stops conflicts between the OS's own Python dependencies and stuff a developer installs for their project. It's still confusing for beginners though — I've run into issues with this myself before I understood how venv worked. Virtual environments solve this properly by giving each project its own isolated package space.

B3. User and Group — Security Context

Python doesn't have its own service user like nginx or postgres does. It just runs as whatever user invoked it. Running Python scripts as root is something you want to avoid — it's a real risk if the script has any external inputs or network exposure.

For web apps the standard practice is creating a dedicated system user:

```
sudo useradd --system --no-create-home --shell /bin/false webappuser
```

This is least-privilege in practice. A Python app running as a restricted user with read-only file access can't do much damage even if something goes wrong.

B4. Service Management

Long-running Python processes get managed through systemd on modern Linux. A basic unit file looks like:

```
[Unit]
Description=Python Web Application
[Service]
ExecStart=/usr/bin/python3 /opt/myapp/app.py
User=webappuser
Restart=always
```

Standard management commands:

- sudo systemctl start myapp
- sudo systemctl stop myapp
- sudo systemctl restart myapp
- sudo systemctl status myapp
- sudo systemctl enable myapp — auto-start on boot

B5. Update Model

Security patches come through the distribution's package manager. When a CVE is found in CPython, the Python security team patches it, distro maintainers backport it, and it comes down through apt upgrade or dnf update as part of normal system maintenance. That's one of the advantages of using the packaged version.

Third-party pip packages are a separate story — those need manual updates. pip install --upgrade package-name, or use pip-audit to scan for known vulnerabilities. pip-audit was built by the Python packaging team themselves so it integrates well.

Part C — The FOSS Ecosystem

C1. Dependencies — What Python Relies On

CPython is mostly C with some Python in the standard library. Its main dependencies:

- GCC or Clang — needed to compile CPython from source
- GNU libc (glibc) — provides basic system calls
- OpenSSL/LibreSSL — powers the ssl module, HTTPS, cryptography
- zlib — compression, used all over the standard library
- SQLite — sqlite3 is built into stdlib, used by a lot of tools
- libffi — ctypes foreign function interface, lets Python call C
- readline — line editing in the REPL

Every one of these is itself open source. Python depends on OpenSSL which depends on glibc which runs on Linux. It's all the way down. This is what people mean when they talk about open-source infrastructure being layered — it's not just metaphor, it's literally how the stack works.

C2. What Python Has Enabled — The Ecosystem

PyPI had over 500,000 packages as of 2024. That number is kind of meaningless on its own but it gives you a sense of scale. The packages that actually matter most are well known at this point:

Scientific / AI

- NumPy — the base array library basically everything else depends on
- pandas — data manipulation, used constantly in data science work
- scikit-learn — ML algorithms, research and production both
- PyTorch and TensorFlow — deep learning, used for almost all current AI research
- Jupyter — interactive notebooks, now standard in scientific computing

Web

- Django — full framework, used by Instagram and Pinterest among others

- Flask — lightweight, good for APIs and smaller services
- FastAPI — newer, built around type hints, very fast

DevOps / Automation

- Ansible — the whole thing is Python
- SaltStack — infrastructure automation
- Fabric — SSH automation

Python has also spawned alternative implementations. PyPy uses JIT compilation and is significantly faster for certain workloads. Jython runs on the JVM. MicroPython runs on microcontrollers with 256KB RAM or less. These exist because the PSF license allows forking — that's not an accident, it's a direct consequence of the licensing choice.

C3. Python in the LAMP Stack and Web

Python isn't in the original LAMP stack (Linux, Apache, MySQL, PHP) but it's become a common PHP replacement in modern architectures. The rough equivalent might be LAPP — Linux, Apache or Nginx, PostgreSQL, Python.

The connection between Python and web servers goes through WSGI (Web Server Gateway Interface), which is an open standard. Gunicorn, uWSGI, and Uvicorn sit between Nginx and the Python app, handling concurrency and process management. It's a bit more setup than PHP's direct integration with Apache, but the architecture is more flexible.

C4. Community and Governance

The Python Software Foundation is a 501(c)(3) nonprofit, set up in 2001. It owns the IP, manages the license, and funds development through sponsorships and membership dues. Importantly it has no commercial owner — nobody can acquire the PSF and take Python in a direction the community doesn't want.

The BDFL model — Benevolent Dictator For Life, which was van Rossum — ran for most of Python's history. It ended in 2018 when van Rossum stepped down after the PEP 572 debate got particularly intense. PEP 572 introduced the walrus operator (:=) and the discussion around it was apparently exhausting enough that he decided he was done making those calls. The Steering Council model replaced it: five people elected by core contributors, rotating.

Development happens on GitHub, discussions on Discourse and the python-dev mailing list, and major language changes go through the PEP system. PyCon happens every year. It's a pretty healthy community structure compared to a lot of open-source projects I've looked at.

Part D — Open Source vs Proprietary (Critical Analysis)

D1. Python vs MATLAB — A Direct Comparison

MATLAB is the obvious comparison for Python in scientific computing. It's been used in academia and engineering for decades, it's specifically optimised for numerical and matrix operations, and it has a massive library of toolboxes for specialised domains.

Dimension	Python (Open Source)	MATLAB (Proprietary)
Cost	Free. No fees at any point.	Roughly ₹1,60,000+ per year for individual license. Institutional

Dimension	Python (Open Source)	MATLAB (Proprietary)
		licenses are negotiated separately and can be very expensive.
Code Transparency	CPython is fully open. Anyone can audit it. CVEs are public.	Closed source. You trust MathWorks entirely for security.
Support	Community support via forums and Stack Overflow. Paid support from Anaconda/Enthought if needed. No SLA.	Official support with SLAs. Dedicated technical team.
Modifiability	Fork it, patch it, redistribute it — full freedom.	None. You can't see the source code.
Governance	PSF nonprofit + thousands of contributors. No single owner.	Entirely controlled by MathWorks. One company makes all decisions.
Performance	Slower for raw matrix work. PyPy and Cython help but there's still a gap.	Very fast for linear algebra and matrix ops. That's what it was built for.
Ecosystem	500k+ packages on PyPI. General purpose.	Good toolboxes but proprietary and costly.
Lock-in	None. Python isn't going anywhere.	High. .m files don't run elsewhere without extra tooling.

D2. Verdict — Would I Recommend Python?

For most things, yes, without much hesitation. Web development, automation, data pipelines, ML, scripting, general-purpose programming — Python is the right call. Zero cost, enormous ecosystem, no license headaches, no vendor dependency. The open-source nature isn't just ideological, it means you're not at risk of a licensing change or a company discontinuing the product.

The one area where MATLAB still makes sense is when you're dealing with legacy scientific code that's been built up over years or decades in MATLAB. Simulink for simulation, code generation for embedded systems — some of that tooling doesn't have a complete equivalent in Python yet. The migration cost for a research group with 20 years of .m files is real. I wouldn't dismiss that.

But for anything new? Python.

Would I contribute to Python directly? Probably eventually. Right now my understanding of the CPython internals isn't deep enough to add real value. But the contribution process is on GitHub, well documented, and the community is welcoming. It's on my list.

Conclusion

I came into this audit thinking open source basically meant free software and left thinking it's one of the more interesting ongoing experiments in how human knowledge and labor can be organised. Python is a good case study precisely because it's been so successful — you can trace a direct line from van Rossum writing code during a holiday break in 1989 to Python

being the dominant language in AI research thirty-five years later. That trajectory happened because of open-source principles, not in spite of them.

The parts of this that stuck with me most were the ethics questions — particularly around companies extracting value from community work without contributing back. I don't think I had a fully formed opinion on that before starting this. I think I do now. The license permitting something doesn't make it a good thing to do.

The technical parts — the Linux directory structure, the systemd service setup, the dependencies — were useful to work through in detail. I'd used Python on Linux plenty but never thought carefully about where things actually lived on the filesystem or how security context worked for deployed apps.

Overall this was a more interesting assignment than I expected going in.

References

- Python Software Foundation. (2024). History and License. https://docs.python.org/3/license.html
- van Rossum, G. (1993). An Introduction to Python for UNIX/C Programmers. NLUUG Najaarsconferentie.
- Open Source Initiative. (2024). The Open Source Definition. https://opensource.org/osd
- GNU Project. (2024). What is Free Software? https://gnu.org/philosophy/free-sw.html
- Raymond, E.S. (1999). The Cathedral and the Bazaar. O'Reilly Media.
- PEP 1 — Python Enhancement Proposals. https://peps.python.org/pep-0001/
- CPython GitHub Repository. https://github.com/python/cpython
- Shotts, W. (2019). The Linux Command Line, 2nd Edition. No Starch Press.
- Python Packaging Authority. (2024). pip documentation. https://pip.pypa.io
- MathWorks. (2024). MATLAB Pricing. https://www.mathworks.com/pricing-licensing.html